What Should the Scheduler Know?

Compiler-Derived Behavioral Hints in a From-Scratch x86-64 Operating System

Teye John Harrison Junior*

Abstract

A CPU scheduler must guess how each program will behave, so it infers that behavior from how the program has run so far. A compiler, by contrast, sees the program *before* it runs. This work asks whether static knowledge from a compiler, delivered to the kernel as a declared behavioral hint, can replace or improve runtime inference, and when it should not be trusted. To answer this, I built the complete stack from scratch. It comprises a preemptive x86-64 kernel in Rust with per-process address spaces, ring-3 isolation and microsecond-resolution per-thread accounting; a native code generator for PlainLang, an English-like programming language, differentially tested against its reference interpreter on 232 programs inside the kernel; a static classifier that labels programs as CPU-bound, I/O-bound or interactive; and a deterministic benchmark harness running under instruction-counted emulation.

Across more than 3,400 runs comparing six scheduling policies, hint-driven scheduling reduced interactive wake-up latency from 7–10 ms under round-robin and multi-level feedback queues to roughly 2 µs. The central result, however, cuts against the premise: a CFS-style fair scheduler using *no* hints matched that latency on hand-designed workloads. Where hints helped measurably was efficiency, reducing context switches by up to 14× at unchanged total completion time. Used as priorities, wrong hints were costly (106 ms latency). A hybrid design that uses hints only to size time slices kept latency near 3 µs even when every hint was wrong. On 150 randomly generated programs, the compiler's classifier matched measured behavior 76% of the time, and every error was in the same direction, underestimating the program's demand. Experiments in the kernel showed that the direction of an error determines who pays for it: an underestimate can delay other programs, whereas an overestimate penalized only the mislabeled program itself.

Predictions were committed to version control before each experiment, and failures are reported alongside successes. Randomized workloads exposed a latency floor caused by timer quantization that no policy avoids, and a regression check uncovered an uncontrolled variable in the author's own measurement apparatus, after which the full dataset was rerun. Results are obtained under emulation with synthetic workloads and an approximation of Linux CFS, and should be read as relative comparisons rather than real-hardware performance.

*AI tools assisted with parts of the implementation and the experiment tooling; the research questions, design decisions and interpretation are the author's. Source, data and every pre-registered prediction: `github.com/JohnTeye0000/aetheros`.